UNINASSAU

Curso de Análise e Desenvolvimento de Sistemas — 2º semestre

Disciplina: Desenvolvimento de Aplicações para Internet

Relatório do Projeto — AV1

Sistema de Cadastro de Alunos

Aluno: **Carlos Gustavo Horta**

Matrícula: **01874179**

Outubro de 2026

1. Qual é a finalidade da aplicação?

A aplicação é um **sistema web de cadastro de alunos**. Pelo navegador, o usuário preenche um formulário com os dados do aluno e o sistema envia essas informações para um servidor feito em Node.js com Express, que guarda os alunos e devolve a lista atualizada.

Com o sistema é possível **cadastrar**, **listar**, **buscar** e **excluir** alunos. O objetivo do trabalho é mostrar, na prática, como o front-end (HTML, CSS e JavaScript) conversa com o back-end (Node.js/Express) por meio de requisições HTTP e dados em formato JSON.

2. Quais alterações foram realizadas no projeto-base?

O projeto-base tinha apenas os campos nome e idade, uma rota POST que guardava o aluno sem nenhuma conferência, uma rota GET e uma lista <ul> montada direto no navegador, com o aviso mostrado por *alert()*. A partir dele, fiz as seguintes alterações:

- **Novos campos:** e-mail e curso, além de nome e idade.
- **Validação no servidor:** todos os campos obrigatórios, idade como número inteiro maior que zero, e-mail em formato válido e bloqueio de e-mail repetido. Quando algo está errado, o servidor responde com status **400** e uma mensagem explicando o problema.
- **Identificador (id):** cada aluno recebe um id único, usado para a exclusão.
- **Nova rota DELETE /alunos/:id** e botão **Excluir** em cada linha, com confirmação.
- **Busca** por nome ou curso, que filtra a tabela enquanto o usuário digita.
- **Tabela** no lugar da lista simples, com **contador** de alunos e aviso quando a lista está vazia ou a busca não encontra nada.
- **A lista passou a vir do servidor** (GET /alunos) ao abrir a página e após cada cadastro ou exclusão. Antes, ao recarregar a página, a lista aparecia vazia mesmo com alunos guardados no servidor.
- **Mensagens na própria página** (verde para sucesso, vermelho para erro) no lugar do *alert()*, e tratamento de erro quando o servidor está desligado (try/catch).
- **Organização das pastas:** HTML, CSS e JavaScript foram para a pasta *public/* (com subpastas *css/* e *js/*), e o servidor entrega somente essa pasta ao navegador.
- **Layout responsivo** (ajusta-se ao celular), uso de *textContent* em vez de *innerHTML* para exibir os dados com segurança, e um **README** explicando o projeto.

3. Quais novos campos foram adicionados?

Foram adicionados dois campos ao cadastro:

- **E-mail** — campo do tipo *email*. No servidor, ele é convertido para letras minúsculas, tem o formato conferido (texto@texto.texto) e não pode se repetir entre alunos.

- **Curso** — uma caixa de seleção (<select>) com os cursos Análise e Desenvolvimento de Sistemas, Ciência da Computação, Sistemas de Informação, Engenharia de Software e Redes de Computadores. Usar uma lista evita erros de digitação.

Além desses, o servidor passou a gerar automaticamente um campo **id** para cada aluno, que não aparece no formulário, mas é usado para identificar o aluno na exclusão.

4. Qual nova funcionalidade foi implementada?

Implementei duas funcionalidades novas que trabalham juntas na lista de alunos:

Exclusão de aluno. Cada linha da tabela tem um botão **Excluir**. Ao clicar, o sistema pede confirmação e, se o usuário aceitar, envia uma requisição **DELETE /alunos/:id**. O servidor procura o aluno pelo id e o remove do array. Se o id não existir, responde com status **404** (não encontrado).

```
app.delete("/alunos/:id", (req, res) => {
    const id = Number(req.params.id);
    const posicao = alunos.findIndex((aluno) => aluno.id === id);
    if (posicao === -1) {
        return res.status(404).json({ mensagem: "Aluno não encontrado." });
    }
    alunos.splice(posicao, 1);
    res.json({ mensagem: "Aluno excluído com sucesso!" });
});
```

Busca de alunos. Acima da tabela há um campo de busca. A cada letra digitada, a tabela é refeita mostrando apenas os alunos cujo nome ou curso contém o texto buscado, sem diferenciar maiúsculas de minúsculas. A busca é feita no navegador, sobre a lista que já veio do servidor, então não precisa de uma nova requisição.

5. Por que essa funcionalidade foi escolhida?

Um sistema de cadastro que só permite adicionar fica incompleto: se um aluno for cadastrado por engano ou sair do curso, não há como corrigir. A **exclusão** resolve isso e completa as operações básicas de um cadastro (criar, listar e excluir). Ela também me permitiu aprender um novo método HTTP (DELETE) e o uso de **parâmetros na rota** (:id).

A **busca** foi escolhida porque, conforme a quantidade de alunos cresce, fica difícil encontrar alguém olhando a tabela inteira. Com o filtro, o usuário encontra um aluno pelo nome ou vê todos os alunos de um curso digitando poucas letras.

6. Como ocorre o envio dos dados do navegador para o servidor?

O envio acontece em etapas:

1. O usuário preenche o formulário e clica em **Cadastrar**.
2. O JavaScript (*script.js*) captura o evento *submit* e chama *event.preventDefault()*, para a página não recarregar.

3. O script lê os valores dos campos e monta um objeto com nome, idade, e-mail e curso.
4. Esse objeto é transformado em texto JSON e enviado ao servidor com *fetch()*, em uma requisição **POST /alunos**.
5. O servidor Express recebe a requisição, valida os dados e, se estiver tudo certo, dá um id ao aluno, guarda-o no array e responde com status **201** (criado). Se houver erro, responde com status **400** e a mensagem do erro.
6. O navegador lê a resposta, mostra a mensagem na tela e, em caso de sucesso, limpa o formulário e pede a lista atualizada com **GET /alunos**.

7. Qual é a função do fetch()?

O **fetch()** é a função do JavaScript que faz **requisições HTTP** a partir do navegador sem recarregar a página. É ele que liga o front-end ao servidor. No projeto, o fetch() é usado três vezes:

- **GET /alunos** — buscar a lista de alunos ao abrir a página e depois de cada alteração;
- **POST /alunos** — enviar um aluno novo;
- **DELETE /alunos/:id** — excluir um aluno.

Como a resposta do servidor demora um pouco para chegar, o fetch() devolve uma *Promise*. Por isso as funções usam **async/await**: o *await* espera a resposta chegar antes de continuar. O bloco **try/catch** trata o caso de o servidor estar desligado.

```
const resposta = await fetch("/alunos", {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify({ nome, idade, email, curso })
});
const dados = await resposta.json();
```

8. Qual é a função do Node.js/Express?

O **Node.js** permite executar JavaScript fora do navegador, no computador que funciona como servidor. O **Express** é uma biblioteca do Node.js que facilita a criação do servidor web e das suas rotas. No projeto, o servidor (*servidor.js*) é responsável por:

- **Entregar a página:** *express.static* envia ao navegador os arquivos da pasta *public/* (HTML, CSS e JavaScript);
- **Entender JSON:** *express.json()* transforma o JSON recebido em um objeto JavaScript, disponível em *req.body*;
- **Definir as rotas da API:** GET, POST e DELETE em /alunos;
- **Validar os dados** antes de guardar, para que dados errados não entrem no sistema mesmo que alguém envie a requisição sem usar o formulário;
- **Guardar os alunos** em um array na memória e **responder** ao navegador com o status HTTP adequado (200, 201, 400 ou 404).

O servidor é iniciado com o comando **npm start** e fica disponível em *http://localhost:3000*.

9. Como os dados são enviados e recebidos em formato JSON?

O **JSON** (JavaScript Object Notation) é um formato de texto para representar dados como pares de chave e valor. Como pela rede só trafega texto, os dados precisam ser convertidos nos dois sentidos:

- **No envio (navegador):** *JSON.stringify()* transforma o objeto do aluno em texto JSON, colocado no *body* da requisição. O cabeçalho *Content-Type: application/json* avisa ao servidor que o conteúdo é JSON.
- **Na chegada (servidor):** *express.json()* converte esse texto de volta em objeto, acessado em *req.body*.
- **Na resposta (servidor):** *res.json()* transforma o objeto ou a lista de alunos em JSON e envia ao navegador.
- **No recebimento (navegador):** *resposta.json()* converte o texto recebido em objeto JavaScript, usado para mostrar a mensagem e montar a tabela.

Situação	Quem envia	Exemplo de JSON
Cadastro (POST)	Navegador	`{ "nome": "Maria Silva", "idade": 20, "email": "maria@email.com", "curso": "Ciência da Computação" }`
Sucesso (201)	Servidor	`{ "mensagem": "Aluno cadastrado com sucesso!", "aluno": { ..., "id": 1 } }`
Erro (400)	Servidor	`{ "mensagem": "Informe um e-mail válido." }`
Lista (GET)	Servidor	`[ { "nome": "Maria Silva", ..., "id": 1 } ]`

10. Quais foram as principais dificuldades encontradas?

A principal dificuldade foi a **configuração do ambiente com Node.js e Express**. Foi preciso instalar o Node.js, entender o papel do *package.json*, instalar o Express com *npm install* e configurar o script *start* para iniciar o servidor com *npm start*. Também foi importante entender que a pasta *node_modules* não deve ir para o repositório (por isso o arquivo *.gitignore*).

Outro ponto foi a **organização dos arquivos na pasta public/**. Ao mover o HTML, o CSS e o JavaScript para lá, foi necessário ajustar o *express.static* para entregar essa pasta (usando *path.join* para montar o caminho corretamente) e corrigir os caminhos *css/style.css* e *js/script.js* dentro do HTML.

Para tirar dúvidas, entender alguns conceitos e revisar o código, utilizei uma ferramenta de inteligência artificial (Claude) como apoio durante o desenvolvimento.

11. O que poderia ser melhorado em uma próxima versão?

- **Banco de dados** (por exemplo, SQLite, MySQL ou MongoDB): hoje os alunos ficam em um array na memória e são apagados sempre que o servidor reinicia.
- **Edição de aluno**, com uma rota **PUT /alunos/:id**, completando o CRUD (criar, ler, atualizar e excluir).
- **Validação também no navegador**, mostrando o erro ao lado de cada campo antes do envio.
- **Login**, para que apenas usuários autorizados possam cadastrar e excluir alunos.
- **Mais campos e filtros**, como matrícula e telefone, filtro por curso e ordenação da tabela.
- **Paginação** da tabela quando houver muitos alunos.
- **Testes automatizados** das rotas da API e **publicação do sistema** na internet.